BSN MARKET

AI Application Development Master Prompt

A multi-vendor Nigerian e-commerce marketplace built with React, Supabase, Vercel and Paystack

Purpose: Use this document as the master instruction for an AI coding tool such as Google AI Studio. It specifies the product requirements, technical architecture, security controls, integrations, testing, and deployment preparation for BSN Market.

Important: *Generated code is not automatically production-ready. Before accepting real orders, verify database access policies, payment signatures and amounts, webhook idempotency, stock handling, refunds, retailer settlements, and order permissions. Use test credentials first.*

1. YOUR ROLE

Act as a senior full-stack software engineer, software architect, UI/UX designer, database engineer, security engineer, and e-commerce product manager.

Your task is to design and build a functional, production-oriented, multi-vendor e-commerce marketplace called BSN Market.

Do not build merely a landing page, static prototype, or frontend demonstration. Build a working application with real database integration, authentication, product management, shopping carts, checkout, order management, retailer dashboards, administration tools, and payment integration.

The application must be suitable for deployment using GitHub, Vercel, Supabase, and Paystack.

I am the business owner and may not have advanced programming knowledge. Explain essential setup steps clearly and provide exact instructions for configuring environment variables, connecting services, running database migrations, testing functionality, and deploying the application.

Do not claim a feature works unless it has actually been implemented and tested.

2. BUSINESS OVERVIEW

Business name: BSN Market

Parent brand: Barrister Street Network (BSN)

Business model: Multi-vendor online marketplace

Primary market: Nigeria

Primary currency: Nigerian Naira (NGN / ■)

Long-term vision: Build a major Nigerian online shopping marketplace where customers can purchase products from multiple independent retailers through one platform.

BSN Market does not initially own inventory, warehouses, or physical retail stores. Independent retailers supply and fulfil orders for their products. BSN Market connects retailers with customers, facilitates transactions, manages marketplace standards, and earns an agreed commission on completed sales.

The marketplace should eventually support expansion into multiple Nigerian cities and additional African markets.

The initial release must prioritise affordability, reliability, security, and ease of operation. Do not assume that the business has substantial startup capital.

Use free or low-cost services where practical, but clearly identify any paid services, usage limits, transaction charges, and potential upgrade requirements.

3. CORE BUSINESS REQUIREMENTS

Build the application around three principal user roles:

1. Customers.

2. Retailers.

3. BSN administrators.

The application must support multiple retailers selling different products through a common catalogue and checkout experience.

Each retailer must manage only their own products, stock, orders, and permitted business information. Customers must be able to browse products, place orders, pay securely, receive order updates, and manage their accounts. Administrators must have central oversight of the entire marketplace.

Implement proper role-based access control using the database and authenticated server-side operations. Never rely exclusively on frontend checks to enforce permissions.

4. REQUIRED TECHNOLOGY STACK

Use the following technology stack unless a genuine technical limitation requires a documented alternative.

Frontend:

• React.

• TypeScript.

• Vite.

• Tailwind CSS.

• A consistent, accessible component system.

• React Router or an equivalent routing solution.

• TanStack Query or an equivalent server-state management approach.

Backend:

• Supabase PostgreSQL.

• Supabase Auth.

• Supabase Storage for product images.

• Supabase Edge Functions where appropriate for privileged operations and payment integrations.

Deployment:

• GitHub for source control.

• Vercel for hosting and deployment.

Payments:

• Paystack for Nigerian payment processing.

• Use Paystack's current official API documentation.

• Verify all payments server-side.

• Never trust frontend payment success messages.

Notifications:

• In-app order notifications.

• Transactional email integration through a suitable email provider.

• Optional web push notifications where practical.

Testing:

• Unit tests for important business logic.

• Integration tests for critical backend operations.

• End-to-end tests for essential shopping and checkout journeys.

Do not use mock data as a substitute for a working production backend. Seed data may be provided for local development, but the application must support real data from Supabase.

5. DESIGN AND USER EXPERIENCE

Create a modern, trustworthy, premium Nigerian shopping marketplace. The interface should feel polished and commercially credible without copying another company's branding.

Design principles:

• Mobile-first.

• Fast-loading pages.

• Clean typography.

• White or light neutral backgrounds.

• Dark text.

• One consistent primary brand colour.

• Clear buttons and calls to action.

• Attractive product photography.

• Simple navigation.

• Accessible contrast.

• Minimal visual clutter.

• Responsive layouts for mobile phones, tablets, and desktops.

Avoid excessive colours, unnecessary animations, oversized banners, complicated navigation, and cluttered dashboards. The marketplace should be easy to use for customers who primarily shop using Android phones.

Use a consistent BSN Market brand identity. Display the BSN Market name clearly. Create a simple text-based logo treatment if no official logo asset is provided. Keep branding replaceable through a central configuration.

Suggested tagline: “Your Market. Your Choice. Delivered.”

Do not invent endorsements, customer testimonials, retailer partnerships, ratings, or sales figures. Display genuine ratings only when supported by verified database records.

All important interface elements must have proper loading, empty, success, and error states.

6. PUBLIC STOREFRONT

Build the following customer-facing pages.

A. Homepage

Include:

• BSN Market branding.

• Search bar.

• Product category navigation.

• Featured products.

• Trending products, based on actual available data.

• Recently added products.

• Retailer promotions.

• Special offers.

• Category cards.

• Product recommendations based on available data.

• Clear links to customer support, delivery information, returns, and policies.

Do not display fabricated statistics or fake product availability. When the database is empty, display a helpful empty-state message rather than broken components.

B. Product catalogue

Customers must be able to:

• Browse all approved, available products.

• Search by product name and relevant attributes.

• Filter by category.

• Filter by price range.

• Filter by retailer.

• Filter by brand where available.

• Sort by price, newest, and relevance.

• Browse products by location when the seller's fulfilment location is relevant.

• View pagination or progressively loaded results.

Only approved products belonging to active retailers should appear publicly.

C. Product details

Each product page should display:

• Product name and images.

• Price in Nigerian Naira.

• Product description.

• Brand, where applicable.

• Available variants such as size, colour, or storage capacity.

• Stock availability.

• Retailer information.

• Estimated delivery information where available.

• Return eligibility.

• Quantity selector.

• Add-to-cart button.

• Buy-now button.

• Related products.

• Verified customer reviews when available.

Validate price, stock, product status, and variant selection on the server during checkout. Never rely on prices submitted by the browser.

D. Shopping cart

Support:

• Adding and removing products.

• Updating quantities.

• Selecting product variants.

• Displaying item subtotals.

• Displaying delivery estimates.

• Showing discounts when valid.

• Displaying the estimated total.

• Revalidating stock and prices before checkout.

Handle products from multiple retailers. If a cart contains items from different retailers, make the fulfilment and delivery arrangements clear. Do not assume all products can be shipped together.

E. Customer checkout

Build a clear checkout process that collects:

• Customer name.

• Email address.

• Phone number.

• Delivery address.

• State.

• City or local government area.

• Additional delivery instructions.

Show an itemised order summary. Calculate all prices on the server using trusted database records. Support Paystack checkout for online payments. Do not expose secret payment credentials in frontend code. Do not mark orders as paid based on query parameters or browser redirects alone.

F. Customer account

Customers should be able to:

• Register and log in.

• Verify their email address.

• Manage their profile.

• Manage saved delivery addresses.

• View previous orders and order details.

• Track order status.

• Access payment references.

• Request cancellation when permitted.

• Request returns or refunds when eligible.

• Leave reviews for eligible completed purchases.

• Contact customer support.

Allow public product browsing without requiring an account. Require authentication before placing an order.

7. RETAILER REGISTRATION AND DASHBOARD

Build a complete retailer onboarding workflow.

A. Retailer registration

Collect:

• Business name.

• Contact person's name.

• Email address.

• Phone number.

• Business address.

• State and city.

• Product categories.

• Business description.

• Available product catalogue.

• Optional business registration details.

• Agreement to marketplace terms and policies.

Store verification status separately from authentication status.

Possible retailer statuses: Pending, Approved, Rejected, Suspended.

New retailers must not be able to publish products or receive live orders until approved by an administrator. Clearly explain the review process to applicants.

B. Retailer dashboard

Provide a dashboard showing:

• Total orders.

• Pending orders.

• Completed orders.

• Cancelled orders.

• Gross sales.

• Commission deductions.

• Net settlement amount.

• Products requiring stock updates.

• Recent order activity.

Use real database queries. Do not display fabricated financial figures.

C. Product management

Retailers should be able to:

• Create and edit their own products.

• Upload multiple images.

• Set descriptions and categories.

• Enter prices and manage stock quantities.

• Define product variants.

• Update availability.

• Submit products for approval.

• Archive their own products.

Product moderation statuses: Draft, Pending approval, Approved, Rejected, Archived.

Only administrators can approve or reject products. Approved products should become publicly visible when the retailer is active and the product is available. Retailers must not be able to change another retailer's products.

D. Order management

Retailers should be able to:

• View orders containing their products.

• Confirm availability.

• Accept or reject fulfilment requests according to clear rules.

• Update preparation status.

• Mark items ready for collection.

• Confirm dispatch.

• View permitted delivery details.

• Report stock problems and fulfilment issues.

Prevent unauthorised access to customer information. Show only the customer information necessary to fulfil each order.

E. Retailer earnings

Provide a transaction ledger recording:

• Order reference.

• Product value.

• Commission rate and amount.

• Refund adjustments.

• Other authorised adjustments.

• Settlement amount.

• Settlement status and reference.

Use a configurable commission structure. Allow different commission rates for different product categories or retailers.

For the initial configuration, use an illustrative default commission of 10%, but make it configurable by an administrator. Never allow retailers to change their own commission rates.

8. ADMINISTRATION DASHBOARD

Create a secure administrative area accessible only to authorised administrators.

A. Overview

Display genuine figures for:

• Registered customers.

• Approved retailers.

• Pending retailer applications.

• Total products.

• Products awaiting approval.

• Orders.

• Completed sales.

• Gross merchandise value.

• Commission revenue.

• Refunds.

• Outstanding settlements.

Clearly distinguish gross merchandise value, commission revenue, and actual profit. Do not label revenue as profit.

B. Retailer management

Administrators must be able to review applications, approve or reject retailers with reasons, suspend and reactivate retailers, view retailer profiles and products, review performance, and inspect fulfilment records.

Suspended retailers must not receive new orders. Implement safe handling of existing orders if a retailer is suspended.

C. Product moderation

Administrators must be able to review, approve, reject, and remove listings; record moderation reasons; manage categories and featured products; and review complaints.

D. Order management

Administrators must be able to view and filter all orders, inspect order details, review payment references, track fulfilment, record cancellations, manage permitted refund workflows, resolve disputes, and review audit logs.

E. Commission configuration

Administrators should be able to configure default, category-specific, and retailer-specific commission rates, effective dates, and supported promotional overrides. Record the applicable commission rate on each order item at the time of the transaction. Historical transactions must not change when future commission rates are updated.

F. Site configuration

Allow authorised administrators to manage store name, logo, contact information, support email and phone, categories, delivery configuration, featured products, promotional banners, return policy, terms of service, and privacy policy.

9. DATABASE DESIGN

Use Supabase PostgreSQL with proper relational integrity. Create database migrations rather than relying on manual, undocumented database changes.

Design the database around these entities:

1. profiles
2. user_roles
3. retailer_profiles
4. retailer_verification_records
5. product_categories
6. products
7. product_images
8. product_variants
9. carts
10. cart_items
11. customer_addresses
12. orders
13. order_items
14. retailer_orders or an equivalent retailer-specific fulfilment entity
15. payments
16. refunds
17. commission_rules
18. commission_ledger
19. retailer_settlements
20. delivery_records
21. product_reviews
22. notifications
23. support_tickets
24. audit_logs
25. product_requests
26. promotional_campaigns

Adapt the schema where necessary to avoid redundant or inconsistent data. Use UUID primary keys where appropriate, foreign keys, unique constraints, check constraints, indexes, and timestamps.

Use numeric or integer minor-unit representations appropriate for monetary values. Never use floating-point arithmetic for financial calculations. Store monetary values in a consistent representation and document it.

Use Africa/Lagos as the default business timezone for displayed dates and times, while storing timestamps in UTC.

Critical order requirements:

• Support orders containing products from multiple retailers.

• Create a parent customer order and separate retailer-specific fulfilment records or an equivalent robust structure.

• Each order item must retain a snapshot of product name, selected variant, unit price, quantity, applicable commission rate, commission amount, retailer, and relevant fulfilment details.

• Historical order records must remain understandable if a retailer later edits or removes a product.

• Use database transactions or appropriately designed server-side operations to keep order creation, stock reservation, and financial records consistent.

• Prevent duplicate orders caused by repeated checkout submissions.

• Use idempotency mechanisms where appropriate.

10. SECURITY AND ROW-LEVEL SECURITY

Security is mandatory. Implement Supabase Row Level Security on every table containing private, financial, administrative, or user-specific information. Policies must enforce permissions in the database, not merely in the UI.

Required access principles:

• Customers can view and modify only their permitted profile information.

• Customers can access only their own orders, payments, addresses, and support records.

• Retailers can manage only their own retailer profile and products.

• Retailers can access only orders and fulfilment records associated with their business.

• Retailers cannot alter payment verification, commission rules, settlement records, or administrator privileges.

• Administrators can access administrative functions only after secure server-side role verification.

• Unapproved retailers cannot publish products.

• Suspended retailers cannot receive new orders.

• Unapproved products cannot appear in public queries.

Never permit users to assign themselves administrative roles. Protect role assignment against privilege escalation.

Do not place a Supabase service-role key in frontend code. Never expose Paystack secret keys, webhook secrets, email-provider secrets, or other privileged credentials in the browser.

Use server-side validation and strict request validation. Protect sensitive operations against SQL injection, cross-site scripting, broken access control, unauthorised file uploads, mass assignment, duplicate payment processing, forged payment notifications, stock manipulation, unauthorised refunds, and commission manipulation.

Use rate limiting where appropriate. Use secure logging that avoids storing passwords, payment secrets, or unnecessary sensitive personal data.

11. PAYSTACK PAYMENT INTEGRATION

Integrate Paystack using a secure server-side payment workflow. Use Paystack's current official API documentation rather than inventing API endpoints.

Required flow:

1. Customer submits checkout.

2. Server validates the cart and delivery details.

3. Server fetches trusted product prices and verifies stock.

4. Server calculates item subtotals, delivery charges, discounts, and totals.

5. Server creates a pending order using a safe transactional workflow.

6. Server initialises Paystack checkout using the correct order amount.

7. Customer completes payment on Paystack.

8. Paystack sends a signed webhook to a secure backend endpoint.

9. Backend verifies the webhook signature.

10. Backend independently verifies the payment transaction with Paystack where appropriate.

11. Backend checks the reference, amount, currency, transaction status, and associated order.

12. Backend marks payment successful only after verification.

13. Backend records commission and retailer settlement obligations.

14. Customer receives an order confirmation.

Implement webhook idempotency to prevent duplicate payment processing. Handle pending, failed, abandoned, successful, refunded, and disputed payments.

Do not fulfil an order merely because a customer reaches a success page. Do not assume Paystack automatically supports marketplace split payments or retailer settlements in the precise way required.

Design commission accounting and retailer settlement records independently of the payment collection mechanism. Before launch, confirm that the chosen Paystack product and merchant arrangement supports the intended payment and settlement workflow.

Never request or store customers' card details directly.

If payment credentials are unavailable, build a clearly identified sandbox or test configuration, document what remains incomplete, and never simulate a real successful payment in production.

12. DELIVERY AND FULFILMENT

Design a delivery system that can start simply and expand later.

Initial capabilities:

• Customer delivery address.

• Retailer fulfilment location.

• Delivery fee configuration.

• Delivery fee quotation.

• Order preparation status.

• Ready-for-pickup status.

• Dispatch status.

• In-transit status where supported.

• Delivered status.

• Failed delivery.

• Cancellation and return workflows.

• Proof-of-delivery reference.

Initially, administrators may configure delivery fees or enter delivery quotations manually. Do not invent delivery rates or promise nationwide delivery where coverage is unavailable.

Design the system so courier APIs can be integrated later. Where an order contains products from multiple retailers, provide separate fulfilment tracking where required.

Do not mark a retailer's order as delivered without an authorised event or administrative action.

13. PRODUCT REQUEST FEATURE

Implement a feature called “Request a Product.” This is important to BSN Market's initial business model.

Customers should be able to request a product that is not currently listed. Collect:

• Product name.

• Product description.

• Category.

• Preferred brand or model.

• Budget range.

• Customer location.

• Optional reference image.

• Contact preference.

The customer should receive a request reference.

Administrators should be able to review requests, search for suitable retailers, contact participating retailers, record quotations, update request status, and notify customers when a suitable offer becomes available.

Possible statuses: Submitted, Under review, Retailer contacted, Quotation available, Customer considering, Converted to order, Unavailable, Closed.

A product request must not automatically become a confirmed purchase. Require the customer to review and accept the quoted product, price, availability, and delivery terms before creating a confirmed order.

Protect customer contact details and uploaded files.

14. EMAIL AND NOTIFICATIONS

Implement useful transactional notifications for:

• Account verification.

• Retailer application received.

• Retailer approval or rejection.

• Product approval or rejection.

• Order placed.

• Payment confirmed.

• Retailer fulfilment accepted.

• Order dispatched.

• Order delivered.

• Cancellation.

• Refund status.

• Product request updates.

• Support responses.

Use a suitable email provider with server-side credentials. If the email provider is not configured, show the missing configuration clearly and preserve the order record.

Do not claim an email has been sent when delivery has not been confirmed by the provider.

Create an in-app notification system for important account and order events.

15. SEARCH ENGINE OPTIMISATION AND PERFORMANCE

Implement:

• Descriptive page titles.

• Meta descriptions.

• Canonical URLs where appropriate.

• Product-friendly URLs.

• Open Graph metadata.

• Product structured data where appropriate.

• Sitemap generation.

• Robots configuration.

• Responsive product images.

• Lazy loading.

• Pagination.

• Database indexes.

• Sensible caching.

• Error monitoring hooks.

Protect private account, checkout, and administrative pages from inappropriate indexing. Ensure public product pages load efficiently on slower Nigerian mobile networks. Do not cache private customer data or administrative responses publicly.

16. LEGAL AND OPERATIONAL REQUIREMENTS

Create editable pages for:

• Terms and Conditions.

• Privacy Policy.

• Retailer Agreement.

• Returns and Refunds Policy.

• Delivery Policy.

• Prohibited Products Policy.

• Customer Support.

Clearly distinguish the retailer supplying a product from BSN Market facilitating the transaction.

Do not invent legal approvals, certifications, or guarantees. Do not claim that BSN Market is the manufacturer or owner of products supplied by retailers.

Prepare the platform for the business owner to review and finalise applicable Nigerian legal, tax, consumer protection, privacy, and payment obligations before launch.

17. TESTING REQUIREMENTS

Write tests for the most important business operations. At minimum, test:

1. Customer registration and authentication.

2. Retailer registration.

3. Retailer approval.

4. Product submission.

5. Product moderation.

6. Public product visibility.

7. Product search and filtering.

8. Shopping cart calculations.

9. Server-side price validation.

10. Stock validation.

11. Order creation.

12. Multi-retailer order handling.

13. Commission calculation.

14. Paystack payment initialisation.

15. Invalid webhook rejection.

16. Valid webhook processing.

17. Duplicate webhook prevention.

18. Order ownership permissions.

19. Retailer data isolation.

20. Administrator permissions.

21. Refund recording.

22. Product request submission.

23. Delivery status changes.

Include tests proving that one retailer cannot access another retailer's private records; customers cannot access another customer's orders; users cannot promote themselves to administrators; and repeated webhook events cannot create duplicate commission entries or retailer settlements.

18. DEPLOYMENT AND ENVIRONMENT CONFIGURATION

Prepare the application for deployment to Vercel. Provide an .env.example containing variable names and placeholder values only.

Expected variables may include:

VITE_SUPABASE_URL=

VITE_SUPABASE_ANON_KEY=

SUPABASE_SERVICE_ROLE_KEY=

SUPABASE_URL=

PAYSTACK_PUBLIC_KEY=

PAYSTACK_SECRET_KEY=

PAYSTACK_WEBHOOK_SECRET=

EMAIL_PROVIDER_API_KEY=

EMAIL_FROM_ADDRESS=

APP_BASE_URL=

Use appropriate public and private environment-variable conventions.

• Only variables intended for the browser may use the VITE_ prefix.

• Never expose service-role credentials, payment secret keys, or email-provider secrets through VITE_ variables.

• Validate required environment variables at startup or before sensitive operations.

• Fail safely when required secrets are missing.

• Do not hard-code production credentials.

• Do not commit .env files containing secrets to GitHub.

• Document which variables belong in Vercel and which are needed for local development or Supabase Edge Functions.

Provide:

1. Project setup instructions.

2. Dependency installation commands.

3. Local development commands.

4. Production build commands.

5. Database migration instructions.

6. Supabase authentication configuration.

7. Storage bucket setup.

8. Row Level Security policies.

9. Paystack webhook configuration.

10. Vercel deployment instructions.

11. GitHub integration instructions.

12. Production smoke tests.

13. Rollback instructions.

Do not claim that external services have been connected or deployed unless that action has actually been completed.

19. DEVELOPMENT WORKFLOW

Work incrementally and preserve a stable, maintainable codebase.

Before implementing features:

1. Establish the project structure.

2. Define the database schema.

3. Identify security boundaries.

4. Design the authentication and role model.

5. Plan the order and payment lifecycle.

6. Define the implementation sequence.

Implement the application in logical stages:

Stage 1: Project foundation, routing, design system, and environment configuration.

Stage 2: Supabase schema, authentication, profiles, and access controls.

Stage 3: Public storefront, categories, product catalogue, search, and product details.

Stage 4: Retailer onboarding, product management, and moderation.

Stage 5: Cart, checkout, orders, and stock management.

Stage 6: Paystack payment processing, webhooks, commission accounting, and settlement records.

Stage 7: Delivery tracking, notifications, reviews, and product requests.

Stage 8: Administration dashboard, reporting, support tools, and operational controls.

Stage 9: Security testing, end-to-end testing, performance improvements, and deployment preparation.

After each stage:

• Run the available checks.

• Fix errors before moving forward.

• Summarise what was implemented.

• List the files created or changed.

• Identify any external setup still required.

• Explain how to test the completed functionality.

Do not silently omit features that are difficult to implement. Do not replace real backend functionality with hard-coded demonstration data.

If output or context limits prevent completing the whole application at once, maintain a clear implementation checklist and continue from the last verified stage. Do not repeatedly regenerate the entire project unnecessarily.

20. IMPORTANT DEVELOPMENT RULES

Follow these rules throughout development:

1. Build a real, functioning application.
2. Keep the code modular and maintainable.
3. Preserve working functionality when adding features.
4. Do not introduce unnecessary dependencies.
5. Never expose private credentials.
6. Never trust prices, payment status, roles, or commission values supplied by the browser.
7. Use server-side authorization for sensitive operations.
8. Use database transactions or safe equivalent workflows for financial and stock operations.
9. Use proper error handling.
10. Do not hide errors behind fake success messages.
11. Do not invent successful external integrations.
12. Do not mark unfinished features as production-ready.
13. Make the application usable on Android phones.
14. Keep operating costs low where practical.
15. Document assumptions and unresolved decisions.
16. Prioritise reliable order fulfilment and customer trust over unnecessary features.

Do not begin by producing only a long explanation or a visual mock-up. Start implementing the actual project.

21. FIRST RESPONSE REQUIRED

Begin by:

1. Confirming the architecture you intend to use.
2. Identifying any essential technical decisions that need to be resolved.
3. Creating the initial project structure.
4. Implementing the foundation of the application.
5. Defining the initial Supabase schema and security approach.
6. Providing clear instructions for connecting the project to Supabase and GitHub.

Then continue through the implementation stages.

The final goal is a functional BSN Market multi-vendor marketplace that can be tested locally, connected to real services, and deployed to Vercel.

PRE-LAUNCH REMINDER

Generated code is not automatically production-ready. Before accepting real orders, verify database access policies, payment signatures and amounts, webhook idempotency, stock handling, refunds, retailer settlements, and order permissions. Use test credentials first and review applicable Nigerian legal, tax, consumer-protection, privacy, and payment obligations.